

Working with Cookies in Express.js

A Comprehensive Tutorial

1. Introduction to Cookies

Cookies are small pieces of temporary data stored in the visitor's browser. Unlike sessions, which store temporary information on the server, cookies store data directly on the client machine.

Common Use Cases

1. **User Authentication** (Login/Logout)
2. **Shopping Cart Storage**
3. **Flash Messages Between Pages**
4. **Remember Me Functionality**
5. **Theme Preferences (Dark/Light Mode)**
6. **Language Preferences**
7. **Form Data Preservation on Validation Errors**

Cookies enable these features by storing small data fragments that persist across page visits.

2. Installing and Using `cookie-parser`

To work with cookies in Express.js, install the `cookie-parser` package.

Installation Command

```
npm install cookie-parser
```

Include and Configure Middleware

```
import express from "express";
import cookieParser from "cookie-parser";

const app = express();

// Using cookie-parser (without secret)
app.use(cookieParser());

// Using cookie-parser with secret key (for signed cookies)
app.use(cookieParser("mySecretKey123"));
```

If using a secret key, cookies can be secured from tampering and treated as "signed cookies."

3. Setting Cookies

Basic Cookie Setup

```
res.cookie("username", "yahoo");
res.send("Cookie has been set");
```

Cookie with Options

```
res.cookie("username", "yahoo", {
  maxAge: 1000 * 60 * 15, // 15 minutes
  httpOnly: true,         // Prevent JS access
  secure: false,          // Set true for HTTPS
  sameSite: "lax",        // strict | lax | none
  signed: true            // For signed cookies
});
```

Explanation of Cookie Options

Option	Purpose
maxAge	Automatic deletion after specified milliseconds
httpOnly: true	Prevents reading cookie via JavaScript; security enhancement
secure: true	Allows access only on HTTPS
sameSite	Controls cross-site access: strict, lax, none
signed: true	Marks cookie as protected by secret key

4. Reading Cookies

Reading Unsigned Cookies

```
const username = req.cookies.username;

if (!username) {
  res.send("No cookie found");
} else {
  res.send("Cookie found: " + username);
}
```

Reading Signed Cookies

```
const username = req.signedCookies.username;
res.send("Signed Cookie: " + username);
```

5. Deleting Cookies

Cookies can be removed manually using `clearCookie`.

Example

```
res.clearCookie("username");
res.send("Cookie has been deleted");
```

6. Practical Route Examples

1. Set Cookie

```
app.get("/set-cookie", (req, res) => {
  res.cookie("username", "yahoo", {
    maxAge: 1000 * 60 * 15,
    httpOnly: true
  });
  res.send("Cookie has been set");
});
```

2. Get Cookie

```
app.get("/get-cookie", (req, res) => {
  const username = req.cookies.username;

  if (!username) {
    res.send("No cookie found");
  } else {
    res.send("Cookie found: " + username);
  }
});
```

3. Delete Cookie

```
app.get("/delete-cookie", (req, res) => {
  res.clearCookie("username");
});
```

```
res.send("Cookie has been deleted");
});
```

7. Signed Cookies Implementation

Setting a Signed Cookie

```
app.get("/set-signed", (req, res) => {
  res.cookie("token", "abc123", { signed: true });
  res.send("Signed cookie set");
});
```

Reading a Signed Cookie

```
app.get("/get-signed", (req, res) => {
  const token = req.signedCookies.token;
  res.send("Signed Cookie Value: " + token);
});
```

8. Important Notes on Cookie Security

1. **Secure cookies do not work in local HTTP development** (they require HTTPS).
 2. **Recommended to always use `httpOnly: true`** to prevent XSS-based theft.
 3. **Use `sameSite=strict` or `lax`** to reduce CSRF risks.
 4. **Use signed cookies for tamper protection.**
-

9. Summary of Workflow

1. Install `cookie-parser`.
 2. Register middleware (with or without secret).
 3. Use `res.cookie()` to create cookies.
 4. Read cookies using `req.cookies` or `req.signedCookies`.
 5. Delete cookies using `res.clearCookie()`.
-